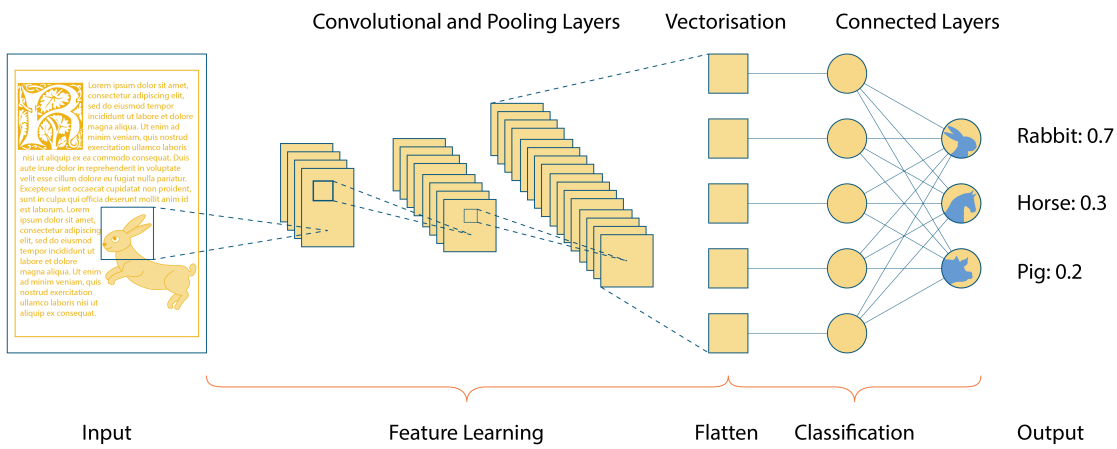
Lesson 09: Vision Classifier

Companion diagram



Convolutional neural network architecture

Image source: https://commons.wikimedia.org/wiki/File:Convolutional_Neural_Network.png

```
Goal
----
Train a small CNN end to end on image-like data and understand the full vision pipeline.

Why this lesson exists
----------------------
The previous CNN lesson taught tensor shapes and layer behavior.
This lesson teaches the whole supervised-learning loop for vision:
- dataset
- dataloader
- model
- train loop
- evaluation

Why synthetic images first
--------------------------
Real datasets add extra complexity:
- downloads
- transforms
- file formats
- larger training times

For a beginner, that can distract from the core ideas.

So this lesson uses a generated dataset with three image classes:
- vertical line
- horizontal line
- diagonal line

That keeps the focus on the pipeline itself.
```

Image shapes

Each generated sample has shape:

```
(1, 16, 16)
```

Meaning:
- 1 channel
- 16 height
- 16 width

The leading `1` is the channel dimension.

When a `DataLoader` makes a batch of 64 such images, shape becomes:

```
(64, 1, 16, 16)
```

How the dataset works

The custom dataset stores:
- a list of images
- a list of labels

Each time `__getitem__` is called, it returns:

```
image, label
```

This is the standard idea behind supervised datasets in PyTorch.

Why noise is added

Without noise, the patterns would be too perfectly clean.
The network could memorize extremely brittle pixel rules.

Noise makes the task a little more realistic.
It encourages the model to learn the underlying structure:
- "this looks like a vertical line"
- not "exactly these pixels are 1"

Model structure

```
Conv -> ReLU -> Pool
Conv -> ReLU -> Pool
Flatten
Linear -> ReLU -> Linear
```

This is a very common beginner CNN structure.

Shape flow through the model

Start:

```
(64, 1, 16, 16)
```

After first conv:

```
    (64, 8, 16, 16)

After first pool:

    (64, 8, 8, 8)

After second conv:

    (64, 16, 8, 8)

After second pool:

    (64, 16, 4, 4)

After flatten:

    (64, 16*4*4) = (64, 256)

After linear:

    (64, 32)

Final logits:

    (64, 3)

Why output size is 3
--------------------
Because there are 3 classes.
The final layer returns one logit per class.

Loss function
-------------
The lesson uses:

    nn.CrossEntropyLoss()

That means:
- model outputs raw logits
- labels are integer class ids

Again, no softmax is needed before the loss.

Train vs test accuracy
----------------------
The script prints both:
- training accuracy
- test accuracy

Why both matter:
- training accuracy tells you how well the model fits seen data
- test accuracy tells you how well it performs on unseen data

If both are high, great.
If train is high but test is low, you may be overfitting.
What `model.eval()` changes
```

```
---------------------------
This script does not use dropout or batchnorm, so the effect is small.
But in real vision models, `eval()` matters a lot.

You should build the habit now:

    train mode for training
    eval mode for evaluation

How this relates to real vision work
------------------------------------
The same structure later extends to:
- MNIST
- Fashion-MNIST
- CIFAR-10
- custom folders of images

The main new pieces later are:
- transforms
- normalization
- augmentation
- larger models
- pretrained networks

Common beginner mistakes
------------------------
1. Forgetting the channel dimension
2. Using the wrong flatten size after pooling
3. Treating logits like probabilities
4. Confusing train accuracy and test accuracy
5. Not understanding that each batch is 4D for images

Exercises
---------
1. Add a fourth class with an anti-diagonal line.
2. Double image size to 32 and recompute the flatten size.
3. Increase noise and observe what happens to test accuracy.
4. Print the logits for one batch and inspect which class score is largest.

Self-check questions
--------------------
1. Why is an image batch 4-dimensional in PyTorch?
2. Why do we add noise to the generated patterns?
3. Why does the final layer have 3 outputs?
4. Why do we measure test accuracy separately from train accuracy?
5. What changes when moving from this toy dataset to a real image dataset?
```

Read the matching Python file and run it while comparing the printed shapes, losses, and outputs against this handout.